

Process States and the PCB

What we ran in class, explained so you can run it again yourself

This walks through the commands we ran live in class, in order, with an explanation of what each one is doing and why. Everything here works on your own machine exactly as it worked on the demo machine, since it has been checked against Arch, Void, Gentoo, Artix, and Xubuntu. Nothing below is destructive, and you are only ever working with processes you started yourself, so do not hesitate to rerun it, change a value, and see what happens.

A note on running this yourself. Almost none of this needs root, since you are only ever touching your own processes. The one exception is a niceness value below zero, mentioned in Step 4, which only root can set.

1. Day 1: Watching Process States and the PCB

This is the demonstration from the day we covered the five state process model and the process control block. Read it alongside those two slides.

Step 1. Start three processes in different states

```
sleep 300 &  
yes > /dev/null &  
sleep 300 &  
kill -STOP %3
```

Here is what each line is actually doing. The first line starts a process that does nothing for three hundred seconds, waiting on a timer, which counts as waiting on a resource, so it lands in the Waiting state right away. The second line starts a process that loops forever, printing the letter y as fast as it can; the output is redirected to `/dev/null` so it does not flood your terminal. Since this process always has work to do, it moves rapidly between Ready and Running rather than ever waiting on anything. The third line starts a second copy of the sleeping process. The ampersand at the end of each line sends the process to the background so your terminal stays free for the next command, and each backgrounded process is given a job number in the order you started it, referred to here as %1, %2, and %3. The last line sends job 3 the STOP signal, which freezes it exactly where it is without letting it run or terminate, so it sits in Linux's stopped state until something tells it to continue.

This leaves you with one process asleep waiting on nothing in particular, one burning CPU in a tight loop, and one explicitly stopped.

Step 2. Show the states before naming them

```
ps -eo pid,ppid,stat,pri,ni,cmd | grep -E "sleep|yes"
```

Here is what each flag is asking for.

- **-e** lists every process on the system, not only the ones started from this terminal.
- **-o pid,ppid,stat,pri,ni,cmd** chooses exactly which columns to display, in this order: **pid** is the process ID, **ppid** is the parent process's ID, **stat** is the current state code, **pri** is the scheduling priority the kernel is currently using, **ni** is the niceness value, and **cmd** is the command and arguments that started the process.

Before reading on, try to guess what the **STAT** column means for each of the three processes. Then map the letters onto Figure 9.7: **S** is Waiting, **R** is Ready and Running (the process switches between the two too fast for **ps** to catch the difference), and **T** is Linux's stopped state, which is not one of the textbook's five states and is worth treating as a real world extension rather than a contradiction.

Step 3. Read a real process control block

Pick the PID of the **yes** process from the output above.

```
cat /proc/<pid>/status
```

Compare **State**, **Pid**, **PPid**, **Threads**, and **VmRSS** to the fields on the Process Control Block slide. The **Vm** prefix stands for virtual memory, and it is the common prefix the kernel uses for a whole family of memory fields in this same file, things like **VmSize**, **VmData**, and **VmStk**, all reported by the part of the kernel that manages a process's virtual memory. **VmRSS** itself is a little confusingly named as a result, since **RSS** stands for resident set size: the amount of physical RAM this process is actually occupying right now, as opposed to the total virtual memory it has reserved. So the field carries a virtual memory prefix while reporting a physical memory quantity. A process can map far more memory than it is using at any given moment, and none of that counts toward **VmRSS** until the process actually touches it and a physical page gets assigned to it. Keep this in mind for Day 2, since it is demand paging that decides when a mapped page becomes resident in the first place.

Before reading on, think about where the saved register contents, the context data field from the PCB slide, would actually live. They are not visible from user space at all: when a process is not running, its registers are saved onto a stack that belongs to the kernel, not anywhere in the process's own memory and not anywhere **/proc** exposes to a user. This is a deliberate choice, not an oversight, and it ties directly back to the memory protection and privileged instruction material from earlier in the chapter: a register can be holding anything the process was working on the instant it was interrupted, so letting any user read another process's saved registers from user space would break the isolation the whole process control block and privilege system exists to guarantee.

Step 4. Watch the dispatcher choose

```
nice -n 10 yes > /dev/null &  
top
```

nice sets a process's niceness, a number from -20 to 19 that biases the scheduler without dictating its decision outright. A higher number makes a process nicer to everyone else, meaning it is asking for less priority, which is why the process started here, at a niceness of 10, gets less CPU time than the other **yes** process running at the default of 0. Only root can push a process's niceness below 0

to make it more aggressive than default, which is itself a small callback to the privileged instruction material.

Inside **top**, press **P** to sort by CPU. You should see the niced process getting a smaller share than the other **yes** process. This is the short term scheduler's fine grained decision, made visible.

Step 5. Clean up

```
kill -CONT %3  
kill %1 %2 %3  
free -h
```

The first line lets the stopped job continue, the second ends all three background jobs, and the third shows a summary of memory and swap in use. Hold onto that last command. We will come back to what it means once we cover memory management.